HAI913I - TP1 - Analyse statique (Universite de Montpellier, 2026-2027)

Analyseur statique de projets Java fonde sur Eclipse JDT Core 3.46.0 : extraction de la structure et des appels, 13 metriques, graphe d'appel pondere, avec trois bonus : visualisation DOT filtree, appels virtuels (CHA) et comparaison avec Spoon.

- **Auteur** : Ay GUNCAVDI - numero etudiant : `21811378` - parcours : `GL` (GL ou ICO)
- **Depot Git** : https ://github.com/ay209/analyseStatique
- **Video de demonstration** : https ://youtu.be/Lr1BsfbacAA

Projet

- **Entree** : le dossier d'un projet Java (sources `.java` ; pas de JAR ni de Maven requis cote projet analyse).
- **Sorties** : texte sur la sortie standard (structure, appels, metriques, graphe) ou Graphviz DOT.
- **Approche** : Eclipse JDT Core (`ASTParser` avec bindings), deux visiteurs (structure, appels) qui produisent un modele de faits immuable ; metriques et graphe sont calcules sur ce modele seul. Bonus realises : C1 (extraction refaite avec Spoon 11.5.0 et comparaison), C2 (Graphviz DOT avec filtres), C3 (option CHA).
- **Dependances** : Eclipse JDT Core 3.46.0 et Spoon 11.5.0 (avec leurs dependances transitives, telechargees par Maven) ; JUnit 5 pour les tests.
- **Limites connues** : voir la section « Difficultes et parties incompletes » du compte-rendu (types locaux et anonymes non comptes, `new C()` n'est pas un appel, appels virtuels suivis au type statique sauf avec l'option `--cha`).

Environnement

Element	Version
JDK	17 minimum (teste avec 17, 21 et 25)
Maven	3.9.6 ou plus recent
Eclipse JDT Core	3.46.0 (telecharge par Maven)
Spoon (bonus C1)	11.5.0 (telecharge par Maven)
JUnit	5.14.4
IDE	aucun requis ; le projet se construit en ligne de commande. Utilise : *Visual Studio Code Version : 1.141.0*
Systeme	*macOS*

Versions exactes utilisees : JDK `25` (`java -version`), Maven `3.9.14` (`mvn -v`).

Construction (depuis une copie propre, sans IDE)

```
git clone https://github.com/ay209/analyseStatique.git tp1 && cd tp1
mvn clean package          # compile, execute les tests, produit target/hai913i-tp1-analyzer.jar
```

`mvn clean verify` execute en plus les tests de bout en bout sur le JAR (`CliIT`).

Utilisation

```
java -jar target/hai913i-tp1-analyzer.jar PROJET ACTION [ARGUMENT]
```

Action	Resultat
`structure`	types, attributs, methodes (avec lignes de corps), totaux
`calls`	appels : ligne, type statique du receveur, cible, interne / externe / non resolu
`metrics X`	les 13 metriques ; `X` (entier >= 0) est le seuil de la question 11
`graph`	graphe d'appel : statistiques et arcs ponderes
`dot`	graphe d'appel au format Graphviz (`... dot > g.dot && dot -Tpng g.dot -o g.png`)
`callers METHODE`	qui appelle `METHODE`
`callees METHODE`	ce qu'appelle `METHODE`
`spoon`	(bonus C1) compare nos resultats JDT a ceux de Spoon 11.5.0

Options (bonus) :

Option	Actions	Effet
`--cha`	`graph`, `dot`, `callers`, `callees`	ajoute les arcs vers les redefinitions connues dans les sous-types (CHA, approximation)
`--package P`	`dot`	ne dessine que les methodes des types du paquetage `P` (sous-paquetages inclus)
`--focus METHODE`	`dot`	ne dessine que `METHODE`, ses appelants et ses appeles

`METHODE` : identifiant complet (`library.model.Item#getTitle()`) ou `Type#methode` (`Item#checkOut`). `PROJET` : dossier du projet ; les sources sont cherchees dans `src/main/java`, sinon `src`, sinon le dossier lui-meme. Les chemins avec espaces sont acceptes (les mettre entre guillemets). `--help` affiche l'aide.

Les resultats vont sur la sortie standard ; erreurs et avertissements sur la sortie d'erreur.

Code	Sens
0	succes (meme si certains fichiers contiennent des erreurs de compilation : elles sont signalees)
1	analyse impossible : dossier inexistant, pas un dossier, aucun fichier Java, methode inconnue
2	arguments invalides : action inconnue, `X` absent ou mal forme (`abc`, `-1`, `5.5`…)
3	erreur interne inattendue (message court, jamais de pile)

Cas d'erreur

```
$ java -jar target/hai913i-tp1-analyzer.jar /nope structure
Le dossier n'existe pas : /nope                      # code 1
$ java -jar target/hai913i-tp1-analyzer.jar src/test/resources/validation metrics abc
Le seuil X doit etre un entier positif ou nul : 'abc'.   # code 2, suivi du message d'usage
```

Avec une erreur de syntaxe dans un fichier, l'outil avertit sur stderr (`Avertissement : 1 erreur(s) de compilation dans 1 fichier(s)...`) et analyse les autres fichiers. Un appel dont la cible est introuvable (dependance manquante) est conserve et marque `[non resolu]`.

Reproduire les points de controle (projet de validation)

Le projet de validation est dans `src/test/resources/validation`.

```
V=src/test/resources/validation
an() { java -jar target/hai913i-tp1-analyzer.jar "$@"; }
an $V structure      # A0/A2 : 14 fichier(s), 15 type(s), 59 methode(s), 493 ligne(s) ; aucun avertissem
an $V calls          # A3    : 94 appel(s), aucun [non resolu]
an $V metrics 5      # B2    : 15 ; 493 ; 59 ; 4 ; 3,93 ; 4,64 ; 1,40 ; Item/Member/LoanService ; ...
an $V metrics 4      # B2    : Q11 -> Book, Item, Member, Catalog, LoanService, TextUtils
an $V graph          # B3    : 59 noeuds, 44 arcs, 46 internes, 48 externes, 0 non resolu
an $V callers Loanable#checkOut   # B3 : LoanService#borrow seulement
an $V callers Item#checkOut       # B3 : (aucun)
an $V graph --cha                 # C3 : 50 arcs (44 sans CHA), dont 6 liens ajoutes vers des redefiniti
an $V dot --focus LoanService#borrow --cha > g.dot   # C2 : puis  dot -Tpng g.dot -o g.png
an $V spoon                       # C1 : memes chiffres que JDT une fois les conventions appliquees
```

Tests et qualite

Commande	Effet
`mvn clean test`	309 tests JUnit (validation, cas limites, proprietes aleatoires, robustesse, encodage, bonus)
`mvn clean verify -Pcoverage`	rapport JaCoCo : `target/site/jacoco/index.html` (seuil 70 % des lignes)
`mvn clean verify -Pquality`	Checkstyle et SpotBugs (rapports, ne casse pas le build)
`mvn clean verify -Pcoverage,quality`	ce que lance l'integration continue (tests, 8 tests de bout en bout `CliIT`, couverture, qualite)
`mvn clean -Pmutation test-compile org.pitest:pitest-maven:mutationCoverage`	tests de mutation PIT : `target/pit-reports/index.html`
`mvn test -Dperf=true`	test de croissance (500 vs 2000 classes)
`mvn test -Drobustness.dir=CHEMIN`	echantillons de robustesse supplementaires

Integration continue : `.github/workflows/ci.yml` (tests JDK 17/21/25, qualite, JAR, image Docker).

Docker

```
docker build -t hai913i-analyzer .
docker run --rm -v "$PWD/src/test/resources/validation:/project:ro" hai913i-analyzer /project metrics 5
```

Depannage

Symptome	Cause probable	Remede
`Le dossier n'existe pas : ...` (code 1)	chemin faux	verifier le chemin ; le mettre entre guillemets s'il contient des espaces
`Aucun fichier Java dans ...` (code 1)	le dossier ne contient pas de `.java`	pointer vers le dossier du projet (celui qui contient `src/`)

Symptome	Cause probable	Remede
`Le seuil X doit etre un entier positif ou nul` (code 2)	`X` absent, negatif ou non numerique	`metrics 5`
`Avertissement : N erreur(s) de compilation...`	des fichiers ne compilent pas	l'analyse continue ; corriger ou ignorer les fichiers signales
beaucoup d'appels `[non resolu]`	une bibliotheque du projet analyse n'est pas disponible	normal sans classpath : ces appels sont conserves et marques non resolus
`Unsupported class file major version` / `release 17 not supported`	JDK trop ancien	utiliser un JDK 17 ou plus recent (`java -version`)
`mvn` ne telecharge pas les dependances	pas d'acces reseau a Maven Central	relancer avec acces reseau ; `mvn -o` ne marche qu'apres un premier build
des tests de validation echouent avec `0` fichier	le dossier `src/test/resources/validation` est vide ou absent	restaurer le projet de validation (il doit contenir `src/library/...`)
avertissements `overlapping classes` pendant `package`	fusion des bibliotheques Eclipse dans le JAR	sans gravite ; utiliser `mvn clean verify` plutot que `verify` sans `clean`

Architecture

```
Main -> cli.Cli -> cli.CommandLine (arguments) -> cli.Analysis -> parse (JDT) -> extract (visiteurs) ->
                                                                    \-> metrics.ProjectMetrics  \-> graph.C
                -> cli.Report (affichage)
```

Seuls `parse` et `extract` touchent JDT ; `model`, `metrics`, `graph` et `cli` n'en dependent pas. Conventions et choix : voir le compte-rendu (`Compte-Rendu.pdf`).

Usage de l'IA

Declare dans le compte-rendu, section « Utilisation des outils d'intelligence artificielle ».